

Resumen Redes de Datos Parcial 2

Capa de Red: Servicios y Distribuciones

Introducción

La **Capa de Red** (3 OSI) se ocupa de llevar paquetes desde el origen hasta el destino final, atravesando múltiples redes interconectadas, teniendo que conocer la topología de estas redes y los routers que debe atravesar en el camino.

Un **router** es el elemento principal de esta capa. Cuando llega un paquete, este se almacena temporalmente en el router, chequea si tiene errores y lo envía al puerto de salida que corresponda, según la tabla interna del router (almacena las direcciones de los destinos asociados a cada puerto).

Un **paquete de datos** es la unidad de información transportada en esta capa (no confundir con trama ya que pertenecen a otras capas del modelo).

Servicio sin conexión

En un servicio de este tipo, cada paquete es transportado por caminos independientes y pueden arribar en forma desordenada al receptor. Estos paquetes se los denomina **datagramas** (compara con un telegrama).

El **protocolo IP** es el más usado en la capa de Red y es el base de Internet, es no orientado a la conexión entre origen y destino final (opera como datagrama).

Servicio orientado a conexión

Primero se establece un **circuito virtual** entre origen y destino, es decir, se selecciona una única ruta por donde enviar todos los paquetes, y se verifica su calidad (igual que en una llamada telefónica). Después se envían los paquetes, con la indicación del circuito virtual a seguir. La desconexión se coordina entre ambos extremos, borrando el circuito virtual. Entonces, la comunicación se hace en 3 etapas: Establecimiento de la conexión – Intercambio de información – Desconexión coordinada.

Comparación entre servicios

Sin conexión es más simple pero complicado de gestionar la calidad de la comunicación, mientras que el orientado a conexión una falla finaliza la comunicación.

SUCESO	SIN CONEXIÓN	ORIENTADO A CONEXIÓN
Establecimiento del circuito virtual	No se necesita	Necesario
Direccionamiento	Cada paquete contiene las direcciones de origen y destino.	Cada paquete contiene sólo el número del circuito virtual.
Información del estado	Los routers no necesitan información extra.	Cada router necesita memoria para almacenar el circuito virtual.
Ruta	Cada paquete tiene una ruta independiente.	Todos los paquetes siguen la misma ruta.
Falla	Se cambia la ruta.	Finaliza la comunicación.
Calidad de servicio	Difícil de gestionar.	Se reservan recursos durante el establecimiento del circuito.
Control de congestión	Difícil de gestionar.	Se reservan recursos durante el establecimiento del circuito.

Distribución broadcasting de mensajes

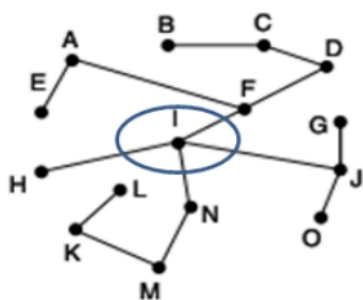
Si es necesario enviar mensajes a todos los usuarios de la red, tenemos varias alternativas para realizar **broadcasting**:

- **Enviar un paquete por usuario**: el más sencillo es enviar un paquete de información a cada uno de los usuarios (lento y poco eficiente).
- **Enviar un mismo paquete a cada router**: consiste en que cada paquete contenga una lista de direcciones destino, así el router que recibe el paquete lo reenvía a cada enlace correspondiente a cada usuario destino.
- **Flooding**: se envía hacia todos los enlaces del router el mismo paquete, utilizando un número de secuencia para no congestionar la red (más rápido pero peligroso para la red).
- **Camino inverso**: si el paquete llegó al router por el **sink tree** entonces se copia y se lo distribuye a los demás puertos de salida del sink tree. Caso contrario, el mensaje se descarta porque seguramente sea un duplicado (forma más eficiente).

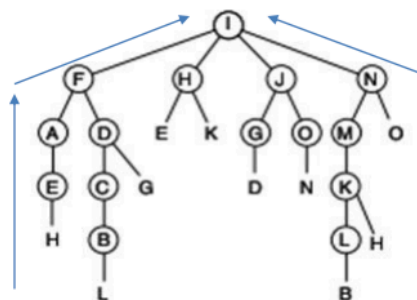
El **Árbol óptimo (sink tree)** es el árbol de enlaces que conecta a un router con cualquier otro router de la red con el menor costo o distancia. Es un subconjunto de los enlaces de la red. Cada router tiene su propio "sink tree", diferente al de los demás.

La forma más eficiente de realizar broadcasting es reenviar el paquete por dicho árbol únicamente si el paquete se recibe por el camino inverso.

Árbol óptimo desde el router I, "sink tree".



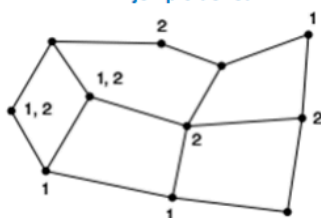
Camino Inverso o "Reverse Path Forwarding" hacia el router I.



Distribución multicasting

Distribución a un grupo de usuarios (multicast routing): se modificó el sink tree para eliminar los enlaces que no pertenecen al grupo y así conseguir la ruta multicasting correspondiente.

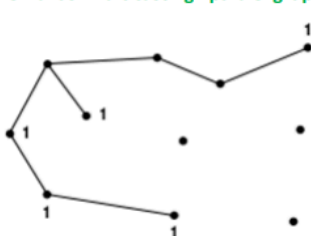
Ejemplo de red.



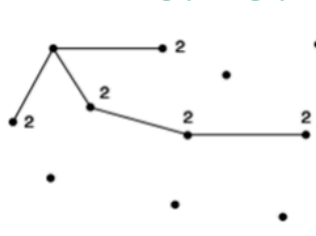
Árbol óptimo para el router indicado.



Un árbol multicasting para el grupo 1.



Un árbol multicasting para el grupo 2.



Distribución multicasting al núcleo del árbol (Core-Based Tree): usa un router raíz para construir el árbol óptimo del grupo. Estos routers raíz se establecen para cada grupo de usuarios. Entonces, se envía el paquete directo hacia ese router raíz, pero apenas el paquete ingresa al árbol, se desparrama por todas sus ramas. Puede ser ineficiente dependiendo cual sea el origen del mensaje multicasting, pero en general es razonable si el nodo núcleo está en el medio de la red.

Distribución anycasting

Distribución a cualquier vecino (Anycast routing): se usa para enviar un mensaje al vecino más cercano, como por ejemplo brindar un servicio como la hora actual, y también se usa para el DNS.

Capa de red: Ruta óptima Origen-Destino

Introducción

La principal tarea de esta capa es dirigir los paquetes desde su origen hasta el destino final. Los algoritmos de selección de rutas de los routers son los responsables del camino completo que sigue un paquete en la red. Estos algoritmos se clasifican **dinámicos** y **estáticos**, los routers emplean algoritmos dinámicos.

En la ruta óptima, para determinar la ruta óptima podemos utilizar como medida el menor número de saltos entre origen y destino. Otro criterio es elegir la ruta con menor retardo, o la más económica. El objetivo de los algoritmos de selección de rutas es encontrar y utilizar la ruta óptima para cada nodo de la red, según el criterio determinado de antemano.

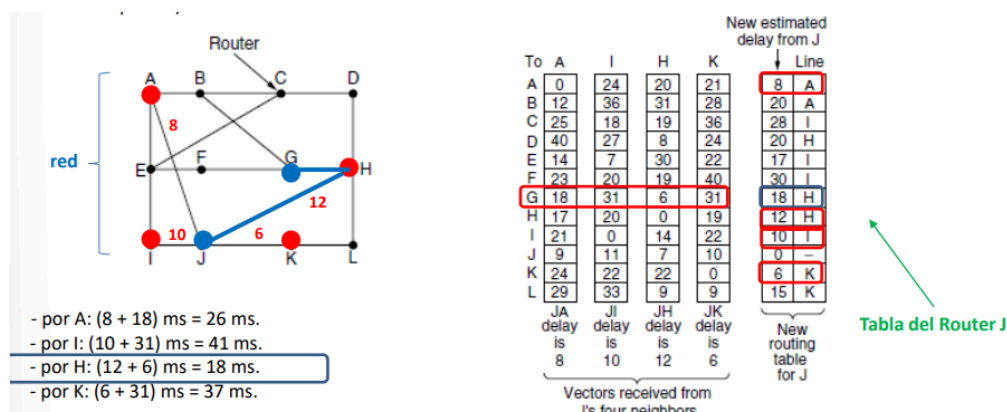
Algoritmo paso más corto

El paso mas corto no es una distancia geométrica, sino que es alguna metrica mas importante para una red, como lo puede ser el retardo promedio, el costo de comunicación, etc. El algoritmo elige salto a salto, el camino de mejor métrica.

Un ejemplo de algoritmo es Dijkstra. VER EJEMPLO PARA ENTENDERLO.

Algoritmo del vector distancia

Se usó en la red ARPANET y en internet con el nombre de RIP (Routing Internet Protocol). Cada router almacena en su tabla interna la distancia hasta cada uno de los routers. VER EJEMPLO (pág 6)



Este algoritmo es muy lento para notificar que un nodo está fuera de servicio. Ya que se necesitaran N actualización de la tabla para actualizar la topología en una red de N nodos en camino lineal. VER EJEMPLO (pág 7).

Algoritmo del estado de enlaces

Se aplicó en ARPANET en 1979, reemplazando el vector distancia. Consta de 5 pasos:

1. Conocer los nodos vecinos y sus direcciones.
2. Determinar la métrica para alcanzar cada nodo vecino.
3. Construir un mensaje con la información recibida de sus vecinos.
4. Enviar un mensaje completo a toda la red y recibir mensajes similares de cada uno de los demás Routers.
5. Analizar toda la información recibida para establecer el mejor camino hacia cada Router.

En el paso 1, apenas se enciende, el Router debe saber quiénes son sus vecinos, por lo que envía un paquete solicitando el nombre de cada vecino. Se usa flooding para que los routers conozcan la topología de red, cada uno chequea los paquetes que le llega si hay nueva información y así enviarla a los demás puertos (sino se descarta). El tiempo de vida del mensaje se decrementa cada segundo, y al llegar a cero, se descarta, evitando bolas de nieve.

En el paso 2, el costo de alcanzar cada vecino, se configura manual o automáticamente. Generalmente se asigna un costo inversamente proporcional a la velocidad del enlace con ese vecino. Otro costo puede ser el costo de retardo del enlace.

En el paso 3, un router recibe un paquete de cada vecino y los distribuye por sus enlaces, dándole conocimiento a los demás vecinos. (se entiende mejor en el ejemplo).

En el paso 4, después de recibir la información de los otros nodos, el router arma su tabla con el costo de cada enlace, donde el costo puede ser distinto dependiendo del sentido (AB no tiene mismo coste que BA, por ejemplo), esta información se envía a toda red.

Por último, se analiza la información y se establece el mejor camino hacia cada router.

Comparado con el vector distancia, acá se requiere más memoria y mayor procesamiento de información en los routers, pero se estabiliza más rápido.

Protocolos de internet

Estos protocolos usan el algoritmo de estado de enlaces:

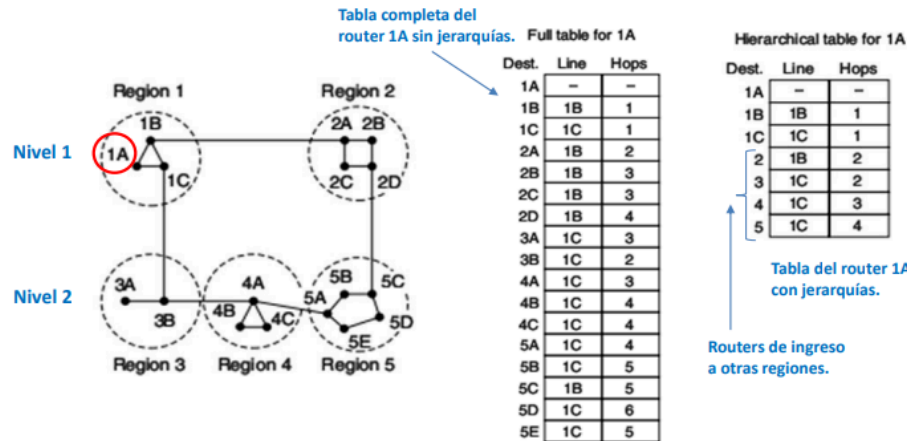
- **IS-IS (Intermediate System – Intermediate System)**: usado en redes DECnet, luego fue adoptado por ISO y modificado para poder usar varios protocolos
- **OSPF (Open Shortest Path First)**: adoptado por IETF, fue adoptando algunas modificaciones similares a las de IS-IS, donde la mayor diferencia radica en que IS-IS puede manejar información de varios protocolos de red, (IP, IPX, AppleTalk), mientras que OSPF carece de esa capacidad.

Selección de rutas por niveles y regiones

A mayor cantidad de routers, mayor trabajo de procesamiento y memoria necesaria para mantener la información actualizada en todos los nodos. Llega un momento en que hay que establecer jerarquías, para simplificar la gestión de la red, porque se torna inmanejable. La

solución es dividir en regiones, donde basta que cada router conozca sólo la topología de su propia región.

En la figura a hay una red con 2 niveles y 5 regiones, en la b está la tabla de 1A sin jerarquías y en la c es la tabla de 1A con jerarquías (se reduce el tamaño porque aparece un solo router de ingreso a cada región externa)



Se descubrió que el número óptimo de niveles para una red de N Routers es el logaritmo natural de N, donde cada Router necesita $e \cdot \ln(N)$ entradas en su tabla interna.

Selección de rutas con usuarios móviles

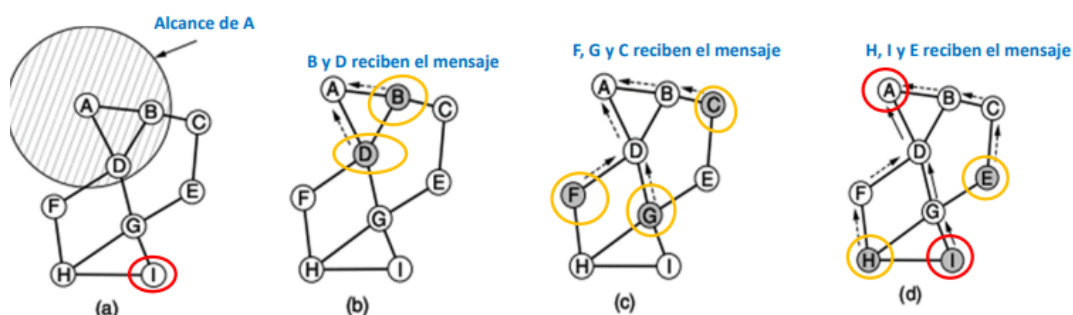
Con **IP Mobility** se agrega una dificultad más, antes de seleccionar la mejor ruta hay que encontrar al usuario final en medio de la red. Esto pasa en internet con los celulares, ya que estos tienen un Home Location, donde si el usuario se ausenta de este se deja a otro agente que funcione como representante (Home Agent) e informa la nueva ubicación. Este agente reenvía los paquetes al celular, cuando este no está en su Home Location.

Selección de rutas en redes AD HOC

En casos extremos, los routers pueden moverse y no estar fijos, como por ejemplo trabajos donde se usen varias notebooks sin access point. Los mismos usuarios son routers.

En lo real, las comunicaciones no son simétricas porque una estación puede tener mayor potencia que otra, pero supongamos que A tiene un mensaje para I, pero no sabe la ruta hacia I. Entonces, envía a sus vecinos un pedido de localización de ruta hasta I y estos hacen lo mismo hasta inundar la red con el pedido (tiene número de secuencia).

Cuando se alcanza el nodo de destino, el pedido se replica hacia atrás, donde los nodos intermedios guardan información del nodo que les envió el pedido, para poder responderle. Cada router incrementa un contador para saber qué tan lejos está del emisor y cuando la respuesta de I llega a A, la nueva ruta queda definida en ambas direcciones.



Se emplea un campo que es **tiempo de vida** en el paquete IP para limitar el tiempo de Broadcast y no saturar la red. De no haber respuesta, se vuelve a enviar el pedido de ruta con un mayor tiempo de vida.

Para mantener activa una ruta preestablecida, cada nodo envía periódicamente un mensaje “Hola” a sus vecinos, que si no responden después de un tiempo, se considera que ese vecino no está disponible y se anulan las rutas que pasaban por el mismo.

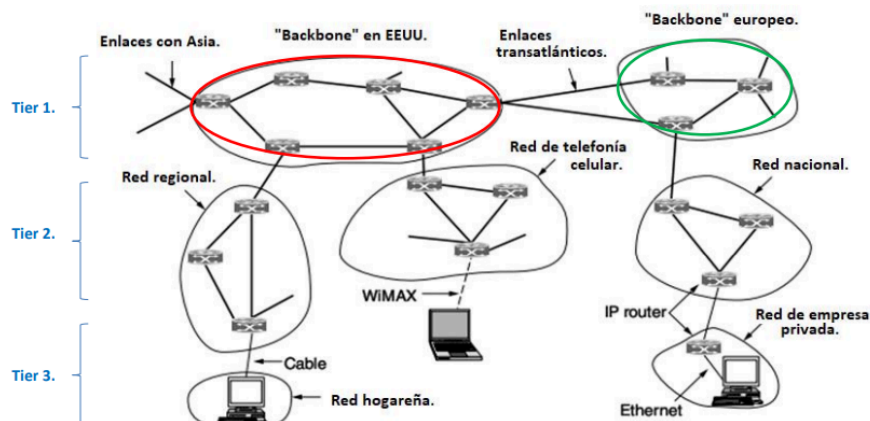
Cada pedido lleva asociado un número de secuencia, así los nodos intermedios pueden eliminar viejas rutas y quedarse con el último número de secuencia.

Para ahorrar recursos se comparte la misma ruta cuando el origen o el destino es un nodo intermedio en el mismo camino y no se hace una nueva búsqueda. Hay algunos algoritmos de este tipo: **DSR (Dynamic Source Routing)**, **AODV (AD HOC, On-Demand Distance Vector)**, **GPRS (Greedy Perimeter Stateless Routing)**.

Capa de red: IPv4

Introducción

Internet es un conjunto interconectado de sistemas autónomos, que no tiene una estructura clara pero si tiene backbones, o anillos de alta velocidad y de distintas jerarquías.



El **Tier 1** es el conjunto de redes más rápidas y de mayor capacidad, que forman los anillos interconectados más importantes de internet.

El **Tier 2** está constituido por los grandes proveedores de internet, que forman redes regionales y se conectan al primer nivel, prestando servicios. Usan grandes data centers donde reside la mayoría del contenido de internet (Claro, Personal, etc).

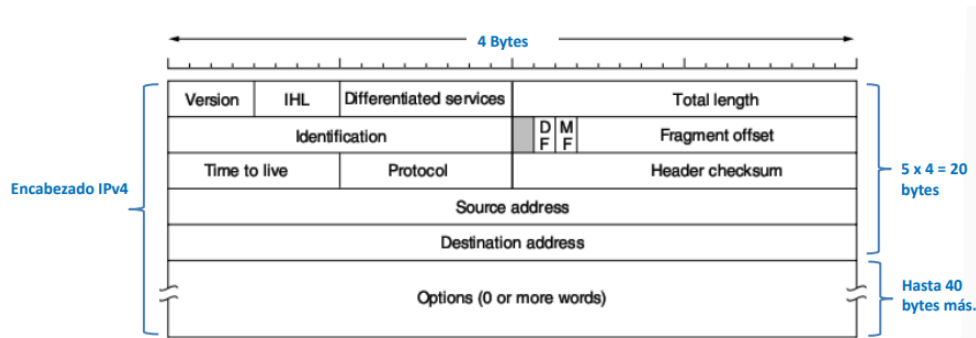
En **Tier 3** esta categoría están las universidades, pequeños ISP y otras organizaciones más pequeñas, que están conectadas a los grandes ISP.

Estos niveles conforman la estructura de internet, que está unida por el protocolo de capa de red **IP (Internet Protocol)**.

Protocolo IPv4

La capa de transporte toma datos de una app, los recorta en paquetes de 1500 bytes, para que sea compatible con Ethernet y se los pasa a la capa de red. Esta los lleva a destino sin importar cuántas redes haya en el camino. En el receptor, la capa de red ordena los paquetes a medida que van llegando y se los da ordenado a capa de transporte.

El encabezamiento del paquete IPv4 tiene una longitud variable entre 20 y 60 bytes, donde los primeros 20 son fijos y se tiene varios entre los que se destacan las direcciones de 32 bits.



El campo **versión** tiene 4 bits, en la mayoría aun indica IPv4 pero hace mas de una década que se definió IPv6.

El campo **IHL (IP Header Length)** indica la longitud del encabezado mediante 4 bits. El mínimo valor es 5 = 0101_2 (20 bytes). Cada incremento va sumando 4 bytes de encabezamiento, por lo que el valor máximo, 15 = 1111_2 , indica el encabezamiento de 60 bytes.

El campo de **Servicios Diferenciados** es de un byte y se utiliza para Calidad de servicio. Se utilizan los primeros 6 bits para designar la clase de servicio, mientras que los otros 2 bits envían información sobre congestiones en la red.

El campo de **Longitud Total** indica la longitud total del paquete IPv4, máximo de $(2^{16} - 1)$ bytes.

El campo de **Identificación** indica el número (2 bytes) de identificación del paquete y permite rearmar la secuencia de mensajes en el receptor.

La zona gris es un bit que no se usa.

El campo **DF** es un bit que si está en 1 indica que no fue fragmentado el paquete.

El campo **MF** es un bit que si está en 1 indica que faltan más fragmentos por llegar del mismo paquete. Se pone en 1, excepto en el último fragmento que lleva 0.

El **Fragment Offset** indica el número del fragmento que viene en el paquete IP. Son 13 bits, que da un máximo de 8191 fragmentos posibles para un mismo paquete IP.

el paquete IP. Son 13 bits, que da un máximo de 8191 fragmentos posibles para un mismo paquete IP.

Time to Live se resta uno con cada router que va pasando el paquete. Tiene 8 bits y por lo tanto, un máximo de 255 saltos, que al llegar a 0 es descartado y se notifica al emisor (evita que queden dando vueltas por la red).

Protocol indica el protocolo de capa 4 que se transporta en el campo de datos (TCP o UDP).

El campo **Header Checksum** detecta errores en el encabezamiento del IPv4. Se calcula en cada router, ya que el time to live cambia con cada salto.

Source address - destination address contiene las direcciones IP del origen y destino.

Hay campos opcionales, útiles para hacer pruebas:

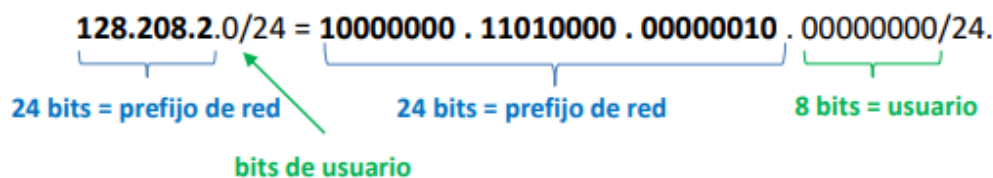
- **Seguridad**, (Security): En la práctica, no se utiliza.
- **Ruta Estricta**, (Strict source routing): Arroja el listado de los routers IP por donde debe pasar el paquete.
- **Rutas Posibles**, (Loose source routing): Lista los posibles routers IP por donde podría pasar el paquete.

- **Identificador de router**, (Record route): Cada router deja indicado su IP y entonces puede conocerse la ruta que siguió el paquete .
- **Tiempo de llegada al router**, (Timestamp): Cada router del camino deja grabado el instante en que el paquete pasó por allí. Es útil para medir tiempos de propagación y retardos.

Direccionamiento IP

Las direcciones constan de 32 bits cada una, dando una capacidad máxima teórica de 2^{32} direcciones. Cada conector de red tiene una dirección de IP. Entonces, un router tendrá un dirección IP diferente en cada una de sus interfaces. En WiFi, cada usuario tiene su dirección IP.

Estas direcciones presentan dos partes distintas, desde la izquierda tienen un prefijo de red (network), que identifica la red (longitud variables y el mismo para todos los usuarios de la red), y los demás bits de la derecha diferencian a cada usuario (host). La network se lo representa con la menor dirección IPv4 de esa red, una barra y la cantidad de bits del prefijo.



La **máscara de subred** es la dirección IP que tiene todos 1 en el prefijo de red y todos 0 en los bits de usuarios. Es útil porque haciendo un AND entre esta máscara y una dirección de cualquier host de esa red, se obtiene el prefijo de la misma.

Direccionamiento por clases

Clase A: tienen prefijo de 7 bits, permite 128 redes clase A y 3 bytes para hosts. Desde **1.0.0.0** hasta **127.0.0.0** (en negrita el prefijo).

Clase B: prefijo de 14 bits, 16384 redes clase B y 2 bytes para host en cada red. Desde **128.0.0.0** hasta **191.255.0.0**.

CLASE C: prefijos de 21 bits, (casi 3 octetos) para redes y puede tener 256 hosts, (1 byte) en cada red. Desde la red **192.0.0.0** hasta la red **223.255.255.0**

CLASE D: se utilizaría para envíos de paquetes multicasts (para grupos de usuarios).

CLASE E: estaba pensada para uso futuro.

Direccionamiento por prefijo (CIDR)

Los prefijos de red facilitan el trabajo de los routers formando “superredes” con una sola entrada en la tabla interna del router, proceso Route Aggregation.

Así se reducen los prefijos IPv4 a unos 200.000 en todo el mundo.

Hay un ejercicio acá, no se que tan importante será así que si quieren leanlo.

Direcciones especiales

0.0.0.0: Se utiliza en los hosts cuando están arrancando, se refieren a la propia red donde se ubica el host.

255.255.255.255 (todos "1"): dirección broadcast para la misma red donde está el host que la envía. Si el prefijo de red es otra dirección cualquiera y la dirección del host son todos 1, indica que es una dirección broadcast en la red correspondiente.

127.X.X.X: utilizadas para bucles dentro de la misma máquina y no salen a la red.

Las direcciones de IP pueden ser públicas, únicas y pueden circular por internet, o privadas, que se usan dentro de una organización sin salir a internet.

Subredes

Una subred es una red dentro de otra red más grande. Se da en grandes organizaciones, donde la red IP se divide en subredes con prefijos diferentes, aprovechando las IP públicas disponibles y facilitando la administración de la red.

Una ventaja es que cada red tiene su prefijo, las tablas de los routers solo necesitan almacenar estos prefijos, y solo necesitan guardar una dirección de cada red. Disminuye la memoria de router necesaria.

La desventaja es que si un usuario cambia de red, deberá cambiar su IP porque debe respetar el prefijo de red de donde está.

Las IP las asigna el ICANN para evitar conflictos mundiales. En cada país, existe un organismo que controla las IP del país. Si se quiere cambiar o modificar las subred, actualizando la máscara, no es necesario contactar al ICANN.

SIGUE UN EJEMPLO.

NAT (Network Address Translation)

Al ser escasas las direcciones IPv4, se usan técnicas para usar las ya existentes.

Una opción es asignar direcciones IP a máquinas activas y cuando se apagan, se reusan las mismas IP en otras máquinas. Así, solo tienen IP las activas.

Se establecieron rangos de IP privadas, de uso libre dentro de una empresa o dentro del hogar:

- Desde 10.0.0.0 a 10.255.255.255/8, que permite hasta $2^{24} = 16.777.216$ usuarios en cada red.
- Desde 172.16.0.0 a 172.31.255.255/12, (hasta $2^{20} = 1.048.576$ usuarios en cada red).
- Desde 192.168.0.0 a 192.168.255.255/16, (hasta $2^{16} = 65.536$ usuarios en cada red).

Las máquinas con IP privadas de una casa, se diferencian por el número del puerto. Estas son direcciones de 16 bits usadas en capa 4, por TCP y UDP. Desde 0 a 4096 se usan para aplicaciones bien conocidas.

En IPv4, NAT soluciona la falta de direcciones pero viola principios del modelo IP, ya que el paquete que sale de una máquina no debería cambiar su IP en su viaje por la red. Sin embargo, cuando se use IPv6 y haya direcciones suficientes, NAT se usará por cuestiones de seguridad.

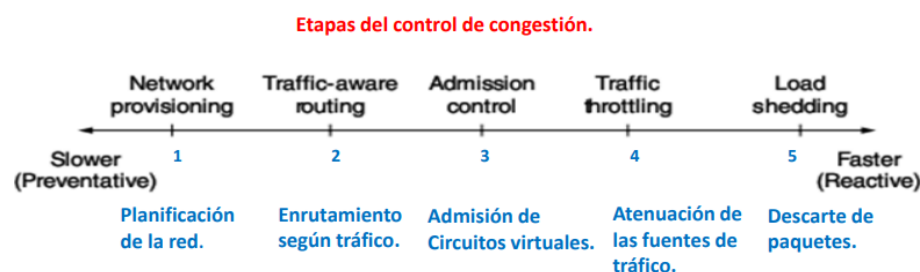
Capa de red: Control de congestión

La congestión es un retardo excesivo, producto del elevado número de paquetes en tránsito a través de la red. La capa de red y de transporte se ocupan de este problema conjuntamente.

Antes de que la red se congestione, se entregan todos los mensajes demandados en un comportamiento lineal, hasta acercarse a la capacidad máxima. Cuando se llenan los buffers de los routers, hay paquetes que se pierden y comienza la congestión, haciendo que los paquetes no lleguen en tiempo y forma, generando paquetes duplicados y que se envíen nuevamente. No es problema de memoria de los routers.

Etapas de la congestión

Para solucionar esto, se aumentan los recursos de la red o se disminuye el tráfico. Distintas técnicas, se aplican en diferentes momentos, siguiendo el proceso de 5 etapas:



Etapa 1: Planificación de red

El primer paso para controlar una congestión es evitarla, mediante una adecuada planificación de la red según el tráfico que debe cursar en el presente y previendo recursos para el tráfico que tendrá en el futuro.

Se deben reemplazar los routers con mucho tráfico por otros de mayor capacidad.

Los enlaces que tienen mucho tráfico también deben ser reemplazados si se ve un incremento en el mismo. Se pueden tener líneas de back up, o alquilar temporalmente enlaces.

Sobredimensionar la red evita la congestión, pero no justifica el costo.

Este proceso de planificación es lento y dura unos meses.

Etapa 2: Selección de rutas según tráfico

Se divide el tráfico por rutas alternativas que son menos transitadas.

En la realidad se debe considerar al tráfico como parte del costo o peso de cada enlace para evitar la congestión. Para ello, se considera el peso de cada enlace con una componente fija (ancho de banda o el retardo de propagación, o el costo económico, etc) y otra componente variable (el tráfico o el tiempo de espera promedio).

El riesgo es que la ruta menos cargada sea la elegida como la mejor, y en consecuencia, el tráfico sea derivado hacia ella. Luego se actualiza la tabla del router, se elige otra ruta por tener menos carga de tráfico, haciendo que el tráfico comienza a concentrarse en la ruta inferior y se genera la **permanente oscilación en el tráfico**.

La solución es permitir que se permita tomar varias rutas con costos similares y cambiar lentamente el tráfico de una ruta a otra (multipath-routing).

Etapa 3: Admisión de circuitos virtuales

Consiste en bajar el tráfico impidiendo la creación de nuevos circuitos virtuales que pasen por la zona congestionada. El pedido de creación de un nuevo circuito virtual debería ir acompañado de la caracterización de su tráfico, en cuanto a su tasa de bits promedio y la forma del tráfico. Más sencillo manejar el tráfico en un stream que el un navegador web.

Etapa 4: Atenuación de las fuentes de trafico

Cuando la congestión es inminente, la red puede pedir a las fuentes de tráfico, que baje la cantidad de tráfico o incluso puede pedirlo la red misma si lo ve necesario para evitar congestión. Periódicamente, los routers envían feedbacks con indicaciones a las fuentes de tráfico para evitar congestiones.

Se usan diferentes técnicas:

- **Aviso hacia atrás:** el router congestionado envía un paquete hacia atrás, a la fuente emisora, y con baja velocidad para no sobrecargar aún más la red, solicitando que disminuya el tráfico hacia ese destino. El paquete puede llevar una marca que indique el problema de congestión, así los routers están notificados y la fuente reduce el tráfico. Luego de un tiempo, puede repetirse el proceso y reducir aún más el trafico.
- **Aviso hacia adelante:** el router congestionado simplemente marca un paquete (choke) y al llegar al destino final, el receptor avisa a la fuente que hay una congestión en camino. Se usa actualmente a internet, es más lento ya que se ejecuta después de que el paquete arriba al receptor y es ejecutado por Transporte (Red solamente marca el paquete).
- **Presion hacia atras:** el router congestionado avisa hacia atrás que disminuyan el trafico todos los router anteriores a él. Cada router reduce el tráfico, almacenando temporalmente los paquetes en exceso mientras que avisa hacia atrás sobre la congestión existente. Es la más rápida, pero le da trabajo a router que tal vez estén sobrecargados.

Etapa 5: Descarte de paquetes

Si las etapas anteriores no controlan la congestión, lo que queda es eliminar paquetes (Load Shedding).

La cuestión es que paquete descartar primero y cuando eliminarlos (lo antes posible es lo mejor). Es conveniente no esperar a que se llene la memoria de un router para empezar a eliminar.

RED (Random Early Detection) es un algoritmo popular y consiste en tomar valores promedios de retardos en las colas de espera de los routers y comenzar a descartar cuando se excede un máximo predeterminado. Aquellos router que cuentan con esto tienen mejor performance que los que simplemente eliminan paquetes cuando su buffer se queda sin espacio.

Dependiendo del tipo de tráfico, se decide qué paquete se elimina primero:

- **Archivos de datos:** es conveniente eliminar los últimos paquetes del archivo transportado, ya que en algun momento seran reenviados nuevamente para completar la transferencia.

- **Streaming:** conviene eliminar primero los paquetes más viejos y conservar los más recientes, no tiene sentido conservar los paquetes viejos si el retardo es grande debido a una congestión.
- **Información de control:** es conveniente eliminar primero los paquetes de datos y conservar los paquetes de los protocolos de control, ya que los paquetes de información son usados en protocolos y algoritmos de los routers.

Capa de Red - Calidad de Servicio

El comportamiento de una red se puede caracterizar por 4 parámetros:

- Velocidad o Bit-rate
- Retardo o Delay
- Variación del retardo o Jitter
- Pérdida de paquetes o Loss

Para transmitir paquetes de manera confiable no es necesario que la red tenga cero pérdidas, ya que pueden recuperarse aquellos paquetes extraviados por medio de retransmisiones o con códigos correctores de errores

Por otro lado una variación del retardo puede suavizarse almacenando datos en el buffer del receptor

Pero nada puede hacerse para solucionar un retardo excesivo o una velocidad insuficiente solo aumentar los recursos de la red

Una manera sencilla de tener buena calidad de servicio es construir una red con capacidad suficiente para transportar cualquier intensidad de tráfico (sobredimensionar/overprovisioning) -> Factible pero caro

Clasificación del tráfico según tasa de bits:

- Tasa de bits constante
- Tasa variable en tiempo real
- Tasa variable en diferido
- Tasa disponible

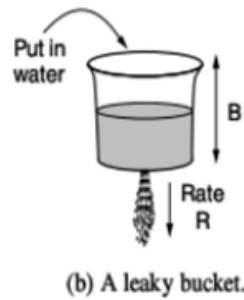
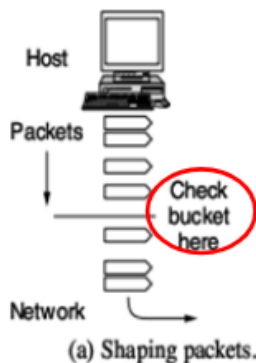
Para asegurar una calidad de servicio adecuada es necesario:

- Saber si la red tiene capacidad disponible o no para incorporar más tráfico.
- Poder controlar y regular el tráfico en la red y además, conocer la “forma” del tráfico.
- Poder reservar recursos de un Router, es decir, reservar tiempos de procesamiento, cantidad de memoria y velocidad de salida

Forma del tráfico - Algoritmo del Balde Agujereado: Imaginemos un balde, como el de la figura. Sin importar la velocidad a la cual se llene el balde, el flujo de salida por el agujero es constante. Si el agua sobrepasa la capacidad del recipiente, simplemente se pierde el exceso de paquetes.

Este modelo puede utilizarse para dar forma al tráfico que llega a una red y fue propuesto por Turner (1989). Los paquetes que arriban al balde tienen que esperar hasta que el balde pueda vaciarse o de lo contrario, son eliminados. Esta técnica permite regular el tráfico que ingresa a la red.

Patrón de tráfico (traffic shaping).



Tráfico promedio R y
capacidad de memoria B .

Algoritmos de Reserva de Recursos (Packet Scheduling Algorithms):

Estos algoritmos asocian recursos de un router a determinados flujos de datos. Esto asegura la calidad del servicio. Se reservan 3 recursos:

- Velocidad de un puerto de salida
- Cantidad de memoria
- Tiempo de procesamiento

Seleccionar la ruta óptima del paquete según los requerimientos de QoS se denomina "QoS Routing". Chen y Nahrstedt, en 1998, dieron una idea para garantizar la QoS: dividir el flujo de datos por diferentes caminos, de modo de encontrar más fácilmente la capacidad necesaria.

Otros algoritmos más complejos fueron desarrollados por Nelakudity y Zhang, 2002. Estos algoritmos tienen en común que toda la ruta entre origen y destino está involucrada en la negociación de recursos sobre un flujo de datos. Para esto debe establecerse un conjunto de parámetros que describa muy bien al flujo de datos en cuestión.

La velocidad de salida depende del algoritmo empleado para las colas de espera en un router

La disciplina FIFO no es siempre óptima para garantizar QoS debido a que los paquetes críticos deben tener prioridad

Algoritmo Fair Queueing: Fue uno de los primeros en proveer reserva de recursos en un router (Nagle 1987). Consiste en que cada router tenga varias colas de espera en cada línea de salida y no una sola

Algoritmo Weighted Fair Queueing (WFQ): Este algoritmo prioriza una cola respecto de otra, dándole más bytes en cada turno. Útil para darle prioridad a ciertos datos

Algoritmo de retardo total: La prioridad es el instante de partida inicial del paquete. Si un paquete se demora mucho en un Router previo del camino pasa a tener mayor prioridad en los siguientes

Reserva de Recursos (RSVP - Resource reSerVation Protocol): Este protocolo es el encargado de hacer las reservas de recursos necesarios para brindar un determinado servicio. El algoritmo comienza en el receptor y va remontando el árbol (Reverse Path Forwarding)

Servicios Integrados:

Un primer intento para desarrollar una implementación de QoS tomó lugar entre 1995 y 1997. El IETF dedicó un gran esfuerzo en diseñar una arquitectura de red para "streaming". Se crearon 20 RFCs y los más importantes se concentran entre los RFC 2205 y RFC 2212. El trabajo se resumió como "Servicios Integrados", sin embargo, su implementación es compleja y no se aplica en la actualidad.

Un método para relacionar un flujo de datos determinado con los recursos de un Router fue dado por Parekh y Gallagher, (1993 y 1994). Se basa en que las fuentes de tráfico estén conformadas por "token buckets".

La velocidad de salida debe estar garantizada en cada uno de los routers del camino. Y el máximo retardo total debe respetarse y, por lo tanto, si algún paquete se demora más de la cuenta en algún router, debe ser priorizado en los routers siguientes.

Estos algoritmos necesitan un establecimiento de recursos previo al envío de los datos y esto puede complicar la red cuando tenemos miles de flujos de datos distintos. Debido a esto no se aplican en la mayoría de las redes.

Servicios Diferenciados (Clases de Servicios):

En 1998 se implementan las RFC 2474, RFC 2475 y otras. Estas se refieren a diferentes "clases de servicios" que pueden ser ofrecidas por una administración que controla los routers de la red.

Salto a salto: La Clase de Servicio se define "por salto", porque depende del trato que cada router del camino le brinde a ese paquete. No está garantizado para todo el camino desde la fuente al receptor.

Sencillez: Este tipo de solución no requiere un establecimiento previo, con reservas de recursos desde emisor a receptor, que consumen tiempo, como es el caso de los Servicios Integrados que vimos antes. Esto hace que estos "Servicios Diferenciados" sean más sencillos de implementar.

La Clase de Servicio depende de cada operador de la red. Como normalmente un paquete es transportado por redes de varios operadores, el IETF establece algunas clases de servicio generales e independientes del administrador de la red.

La RFC 3246 trata la implementación de 2 clases de servicios diferentes: regular y especial. Mientras que la mayoría de los paquetes tiene un servicio regular, unos pocos paquetes son tratados de manera especial, con pequeños retardos, jitter y pequeñas pérdidas.

Los paquetes que tienen este servicio especial son marcados en la fuente de origen como tal (o en el primer router). Estos paquetes pasan físicamente por la misma, aunque son diferenciados en forma lógica, y los routers los colocan en colas de espera de mayor prioridad que el tráfico común.

La RFC 2597 define una alternativa más compleja para tratar Clases de Servicios. Esta solución prevé 4 prioridades de tráfico diferentes (por ej oro, plata, bronce y carbono). Y por otro lado, se definen 3 modos de descartar paquetes congestionados: alto, medio y bajo. Combinando las 4 prioridades con los 3 modos de descarte resultan 12 servicios diferenciados.

Capa de Red - Protocolos de Control

Internet Control Message Protocol (ICMP):

Además del protocolo IP existen varios protocolos cuya función es controlar el tráfico de los datos. Los mensajes enviados por los protocolos de control, en general, viajan desde un router hacia otro router, mientras que los datos viajan desde un host a otro host

Cuando sucede algo inesperado en internet, el router envía un mensaje en protocolo ICMP al origen, encapsulado en un paquete IP. Existen una docena de mensajes ICMP.

Algunos mensajes ICMP son:

- Destino inalcanzable: Este mensaje se utiliza cuando el router no logra alcanzar al destino
- Tiempo excedido: Se envía al origen cuando el paquete es descartado porque llegó a 0 su campo "Time to live"
- Problema en parámetro: Se emplea al detectarse un valor incorrecto en el encabezamiento del paquete IP
- Bajar tráfico de fuente: Se utilizaba para que una fuente origen disminuya la velocidad de transmisión, en caso de congestión. Hoy esta técnica de "aviso hacia atrás" está en desuso.
- Ruta alternativa: indica que existe otra ruta mejor hacia el destino en cuestión.
- Eco: se emplea en el comando "ping" para saber si un host está operativo, y en ese caso el router responde con un "echo replay"

Address Resolution Protocol (ARP):

Si una PC en una red Ethernet, tiene que enviar un mensaje a una cierta dirección IP, necesita conocer la dirección MAC de Ethernet, (en capa 2) del host destino, asociada a esa IP, para poder encaminar su mensaje. Esto se consigue mediante el protocolo ARP, que está descrito en la RFC 826.

Dynamic Host Configuration Protocol (DHCP):

La dirección IP de un host puede configurarse a mano o por medio de un servidor DHCP. Al encenderse una PC envía un broadcast solicitando una dirección IP, utilizando el protocolo DHCP. El servidor DHCP le responde asignando una IP. La dirección IP así obtenida tiene un tiempo de uso limitado y luego expira o se renueva si la PC continúa activa en la red. Se denomina IP dinámica.

El DHCP está especificado en las RFC 2131 y 2132

Multiprotocol Label Switching (MPLS):

El funcionamiento de las redes MPLS se aproxima al de las redes de conmutación de circuitos telefónicos o de circuitos virtuales

Estas redes agregan una etiqueta al paquete IP y el encaminamiento se efectúa más por esta etiqueta que por la dirección IP destino. Así, una tabla asocia la etiqueta con el puerto de salida correcto, y el encaminamiento es más sencillo y más rápido, ya que no hay mucho procesamiento. MPLS comenzó a utilizarse por algunos fabricantes y luego el IETF lo registró en la RFC 3031 y algunas otras RFC.

Los routers IP no fueron pensados para llevar una "etiqueta", como lo requiere MPLS.

Para esto se emplean routers especiales llamados Label Switched Router (LSR). Estos equipos llevan un encabezamiento con 4 bytes (32 bits) para MPLS entre la capa 2 y la capa 3

El encabezamiento MPLS lleva una etiqueta de 20 bits, luego un campo para "QoS", después un bit que indica si existe más de una etiqueta, ya que pueden ser varias, y finalmente el campo "Time to Live" para impedir que el paquete circule indefinidamente por la red

Open Shortest Path First (OSPF):

Internet está constituida por muchas redes privadas interconectadas que se denominan Sistemas Autónomos (SA). Dentro de cada SA se utiliza un protocolo de encaminamiento interno. Uno de los más utilizados es OSPF

Este protocolo utiliza el algoritmo de "estado de enlaces". En 1990 el IETF estandariza OSPF basándose en el protocolo de estado de enlaces de ISO llamado IS-IS (Intermediate System to Intermediate System). Hoy en día, los routers de los principales fabricantes soportan tanto OSPF como también IS-IS.

En estos protocolos, los caminos con igual distancia son registrados y se utilizan para balancear cargas, repartiendo el tráfico para evitar oscilaciones de tráfico. El balance de carga se denomina Equal Cost Multi Path (ECMP).

Para manejar redes muy grandes, OSPF permite la subdivisión de un sistema autónomo en varias áreas o regiones, que no deben solaparse entre sí.

Cada región es una red y los routers dentro de una región se denominan routers internos.

La topología de la región no se conoce externamente, aunque sí sus puntos destino.

Esto facilita la tarea de los routers para el enrutamiento de paquetes.

El área central se denomina "Backbone" o área cero, y sus routers son "backbone routers".

Todas las demás áreas se conectan al backbone. Los routers que conectan a dos o más áreas se denominan "routers frontera" y deben pertenecer al backbone.

Los routers frontera concentran la información de distancias dentro de un área y la distribuyen en otras áreas. Pero no informan sobre la topología total del área para simplificar el protocolo. Estos routers calculan el paso más corto dentro de cada área por separado, con la información obtenida en cada área.

Cuando un router se enciende, envía mensajes "Hello" en todas sus líneas de salida para saber quiénes son sus routers vecinos. Sin embargo, como es muy ineficiente dialogar con todos los routers vecinos, se elige un router en cada LAN como el "designated router", y todos los routers de la LAN dialogan con él. También se designa un "router backup", que concentra la información y lo reemplaza en caso de que el router elegido falle y quede fuera de servicio.

En operación normal, periódicamente cada router envía su información actualizada al router elegido, es decir envía un "Link State Update". Éste responde con un "acknowledge" y según el número de secuencia del paquete sabe si el mensaje es nuevo o viejo

Exterior Border Gateway Routing Protocol (BGP):

Entre SA diferentes se emplea BGP. Se dice que BGP es un protocolo entre dominios diferentes. BGP tiene que poder manejar políticas diferentes que pueden existir en SA diferentes

Las rutas óptimas de encaminamiento son privadas, ya que contienen información sensible del negocio de cada SA y no se dan a conocer a los demás SA.

Es común que una compañía tenga más de un ISP, de modo que si un ISP falla puede acceder a internet mediante otro ISP. Esto se llama “multihoming” y es soportado por BGP. Además, un SA podría traficar datos de otro SA y viceversa, sin costos bajo ciertas condiciones. Este tráfico recíproco gratuito se llama “Peering”.

El protocolo BGP además de considerar el costo de cada ruta hacia un destino, tiene en cuenta el camino que va recorriendo hacia el destino. Los protocolos que operan con estos algoritmos se denominan “Protocolos del vector de paso” o Path Vector Protocol.

Los routers BGP establecen enlaces TCP entre ellos para que la comunicación sea muy confiable.

Los routers BGP encaminan el tráfico de datos en un sentido, y transmiten en sentido contrario los detalles del camino recorrido. De este modo, puede detectarse bucles porque llegarían a un mismo router indicadores de caminos con destinos en común. Si se detectara un bucle, se descarta la ruta.

Al pasar de un SA a otro, el BGP almacena el paso recorrido. Por lo tanto, la ruta utilizada es normalmente la de menor costo económico para el SA.

Capa de Red - Protocolo IPv6

El IETF comenzó a trabajar en una nueva versión de IP en 1990 con el fin de lograr las siguientes mejoras:

- Soportar más de mil millones de direcciones efectivas de hosts (10^9 o 2^{30})
- Hacer posible que un host cambie de lugar físico, conservando su dirección IP.
- Reducir el tamaño de las tablas de enrutamiento en los routers.
- Simplificar el trabajo de los routers para que trabajen con mayor velocidad.
- Proveer mayor seguridad en autenticación y privacidad.
- Poner el foco de atención en el tipo de servicio, especialmente con datos en tiempo real.
- Mejorar el servicio multicasting permitiendo rangos de direcciones específicos para ello.
- Permitir evoluciones futuras del mismo protocolo.
- Permitir que las viejas versiones del protocolo coexistan con las nuevas.

En principio se lo llamó Simple Internet Protocol Plus (SIPP) y luego se denominó IPv6.

Todo el detalle se encuentra en las RFC 2460 hasta 2466.

Las direcciones IPv6 tienen 16 bytes de longitud (128 bits) lo cual permite armar hasta $2^{128} = 3,4 \times 10^{38}$ direcciones.

Para lograr un procesamiento más veloz por parte de los routers, tiene un encabezamiento más simple que IPv4, con el agregado de campos opcionales.

Presenta mejoras importantes en seguridad, que luego también se implementaron en IPv4.

Mejora la QoS, implementando varias clases distintas de servicio.

En líneas generales, IPv6 no es compatible con IPv4, aunque sí es compatible con otros protocolos auxiliares que lo acompañan en el modelo de capas.

Formato IPv6:

- Versión: Este campo de 4 bits lleva 0110 en IPv6 y 0100 en IPv4. Los routers examinan este campo para saber qué paquete están transportando. Algunos routers pueden obviar este paso debido al encabezado de capa 2.

- Calidad de Servicio: Distingue diferentes clases de servicios e indica congestión, como en el caso de IPv4.
- Etiqueta del flujo de datos: Este campo permite indicar si un grupo de paquetes tiene requerimientos similares en cuanto al flujo de datos, es decir, el nivel de retardo admisible, jitter, tasa de bits y descarte de paquetes
- Longitud de carga: Indica la longitud del campo de datos, es decir, cuántos bytes siguen después del encabezado.
- Próximo Encabezamiento/ Protocolo: Indica si hay campos adicionales en el encabezado. Si no hay campos adicionales, entonces expresa el tipo de protocolo transportado en el campo de datos.
- Límite de saltos: Indica que, si se sobrepasa el número de routers establecidos, el paquete se descarta. Similar al "Time to Live" de IPv4

Direcciones Origen y Destino: Estos campos de direcciones son de 16 bytes de longitud y se expresan distinto que en IPv4. En IPv6 se indican con 8 números hexadecimales de 2 bytes cada uno, separados por dos puntos.

Encabezamientos adicionales:

En algunos casos es necesario añadir a IPv6 algunos campos utilizados en IPv4 y para esto se agregan los "Extension Headers". Se definieron 6 tipos diferentes. Algunos tienen formato fijo, otros pueden tener longitudes variables, que van definidas por tres parámetros: Tipo, longitud y valor del encabezamiento extendido.

- Tipo: Es solo un byte. Los 2 primeros bits definen la acción a tomar por el router: Dejar pasar el paquete; descartar y enviar un ICMP hacia atrás; o descartar y no enviar ningún mensaje ICMP. Además, indica si el encabezamiento adicional se extiende más de 8 bytes, que es la longitud por defecto
- Longitud: Tiene un solo byte e indica la longitud exacta del encabezamiento adicional, que puede variar entre 0 y 255 bytes
- Valor: Indica más información, que puede extenderse a 255 bytes, como máximo

Los 6 tipos de encabezamientos adicionales y opcionales.

Extension header	Description	
Hop-by-hop options	Miscellaneous information for routers	Información para los routers.
Destination options	Additional information for the destination	Información sobre el destino.
Routing	Loose list of routers to visit	Lista posible de routers a encontrar en el camino.
Fragmentation	Management of datagram fragments	Gestión de fragmentos.
Authentication	Verification of the sender's identity	Autenticación del emisor.
Encrypted security payload	Information about the encrypted contents	Información sobre contenidos encriptados.

Comparacion con IPv4:

En IPv6 desaparece el campo IP Header Length (IHL), debido a que la longitud del encabezado es fija en IPv6. También desaparece el campo Protocol ya que se utiliza el mismo campo Next Header para indicar el tipo de protocolo que se transporta. También desaparecen todos los campos relativos a la fragmentación de paquetes. En IPv6 se utiliza un encabezamiento adicional para la fragmentación y emplea el "Maximum

transmission unit (MTU) path” para regular el tamaño máximo de los paquetes. Si el paquete es mayor al largo máximo aceptado por la red, el router lo devuelve al origen.

Se elevó la longitud mínima del paquete IP, pasando de 576 bytes a 1280 bytes

Desaparece el campo “Checksum” en IPv6, para darle mayor velocidad a los routers y porque tanto la capa 2 como la capa 4 tienen sus propios controles para detección y corrección de errores. Por lo tanto, tampoco hay campo de número de secuencia.

Movilidad en IPv6:

Por medio de paquetes ICMP, el dispositivo móvil se entera de que está en otra red, porque advierte que no coincide la dirección IP con la de su router habitual. Entonces el host solicita y obtiene una nueva dirección IP por DHCP. También utiliza ICMP para saber quién es su representante allí (Foreign Agent). Después informa las respectivas direcciones IP a sus representantes, “Home Agent” y “Foreign Agent” y ya está en condiciones de comunicarse por internet en su nueva ubicación. El “Home Agent” intercepta los paquetes destinados al host que no está, por un mecanismo Address Resolution Protocol (ARP). Cuando el router pregunta entonces por el host, el “Home Agent” responde con la dirección Ethernet del host que ya no está. Este mecanismo se denomina “proxy ARP”

Los ISPs normalmente chequean por seguridad, las direcciones IP de origen para saber si se condicen con el enrutamiento. Cuando el mensaje proviene del “Home Agent” hacia el host viajero, transporta una dirección IP origen distinta, por lo tanto, podría ser descartado. Para evitar esto, el “Home Agent” podría utilizar su IP como origen del mensaje para que éste no se descarte. La desventaja es que el mensaje viaja un recorrido mayor por la triangulación. Además, los mensajes entre el host y los “Agents” van encriptados por seguridad, para evitar que un tercero se apodere de los mensajes que deberían reenviarse a un determinado dispositivo móvil IP.

La característica de la movilidad en IPv6 está especificada en la RFC 3775. La idea es la misma que en IPv4, sólo que está optimizada para evitar la triangulación y que el mensaje viaje directamente entre el host móvil y su destinatario, sin pasar por los “Agents”.

Una innovación es que se define un segundo nivel de movilidad, cuando el que viaja es el router. Este proceso se denomina “Network Mobility” y está definida en la RFC 3963.

Capa de Red - Interconexión de Redes

Antes de conectar redes diferentes, debemos resolver las diferencias entre ellas.

Cerf y Kahn propusieron en 1974 lo que ahora conocemos como protocolos TCP/IP.

Los Routers, que interconectan redes y trabajan en capa 3, interpretan el protocolo IP{

DISTINTOS ASPECTOS EN QUE DOS REDES PUEDEN DIFERENCIARSE

Item	Some Possibilities	
Service offered	Connectionless versus connection oriented	- SERVICIO ORIENTADO A CONEXIÓN O NO.
Addressing	Different sizes, flat or hierarchical	- DIRECCIONAMIENTO POR PREFUJOS O NO.
Broadcasting	Present or absent (also multicast)	- CON O SIN BROADCASTING.
Packet size	Every network has its own maximum	- PAQUETES DE DISTINTOS TAMAÑOS.
Ordering	Ordered and unordered delivery	- ENTREGA ORDENADA O NO.
Quality of service	Present or absent; many different kinds	- DIFERENTES QoS.
Reliability	Different levels of loss	- DISTINTOS NIVELES DE PÉRDIDAS DE PAQUETES.
Security	Privacy rules, encryption, etc.	- REQUISITOS DE SEGURIDAD.
Parameters	Different timeouts, flow specifications, etc.	- PARÁMETROS DIFERENTES, FLUJOS DISTINTOS.
Accounting	By connect time, packet, byte, or not at all	- TARIFA POR TIEMPO, O POR TRÁFICO, ETC.

IP no es el único protocolo para llevar datos en capa 3. Varias empresas han creado los suyos propios en un intento por posicionarse mejor en el mercado. Ej: IPX de Novell, AppleTalk, SNA de IBM, etc.

Algunos routers se denominan multiprotocolo porque pueden traducir un protocolo en otro. El protocolo IP también tuvo gran éxito porque funciona como un mínimo denominador común, que requiere poco de la red. El servicio que brinda es “best effort”

Comunicación Túnel:

La “comunicación túnel” es un caso especial que se da cuando dos redes iguales en capa 3 se conectan por intermedio de una tercera, cuyo protocolo de capa 3 es distinto al de las otras dos.

Es una excepción dentro del modelo de capas, porque se anida un paquete de capa 3 en otro paquete en la misma capa.

Rutas Multiredes:

Si dos redes pertenecen a organizaciones distintas, mientras una red minimiza el retardo, la otra, tal vez, minimiza el costo de transporte. Cada organización es un SA con sus propias políticas y criterios para administrar la red. En estos casos, es necesario agregar otro nivel de protocolo para selección de rutas para que ambas redes operen con un mismo protocolo “interredes”, como por ejemplo, BGP

Al transportar tráfico de datos entre distintas organizaciones, los ISP establecen acuerdos comerciales entre ellos. Si el tráfico es internacional, también hay leyes distintas que respetar según el país

Fragmentación de Paquetes:

La máxima longitud de los paquetes es otro problema para interconectar redes y depende de varias razones.

- Fragmentación transparente: Este método fragmenta los paquetes en la red que así lo requiera, y se rearma el paquete grande apenas se llega al final de dicha red. Si

más adelante hay otra red con restricción del tamaño máximo, se vuelve a fragmentar el paquete

- Fragmentación no transparente: Este método es más eficiente, una vez que se fragmenta el paquete de datos, sigue así, fragmentado hasta su destino final y recién allí se rearma.
- Fragmentación en IP: El paquete IP lleva en el encabezado un número "identification" que identifica el paquete. Otro campo "fragment offset" con 13 bits identifica el primer fragmento del mismo paquete. Finalmente en el campo "More Fragments" (MF) se coloca un bit en 1 para indicar que es el último fragmento del paquete. Si el paquete no está fragmentado se coloca un 0 en el campo "Don't Fragment" (DF)

El método que se implementa en la internet moderna con IPv4 se denomina "Path MTU Discovery" y consiste en que el router devuelva el paquete a su origen cuando se detecta una longitud mayor a la máxima que permite la red

Protocolos Multicasting:

Una comunicación en internet puede estar dirigida a muchos receptores simultáneamente y desde un solo emisor

IP soporta multicasting empleando direcciones clase D, reservadas para estos casos, y que comprenden 28 bits.

Las direcciones 224.0.0.0/24 están reservadas para multicasting en una red local

Las direcciones se transmiten y son reenviadas por los routers hacia dentro de la LAN, no afuera. Si un host afuera de esa LAN también pertenece al grupo multicast en cuestión, debemos emplear un protocolo multicast, como IGMP. En este caso los routers multicast preguntan quienes están en el grupo multicast, utilizando la dirección 224.0.0.1 y los hosts del grupo le responden. IGMP está especificado en la RFC 3376.

Para construir un "Multicast Spanning Tree" se utiliza el protocolo "Protocol Independent Multicast" (PIM). Si el grupo multicast se extiende más allá de un sistema autónomo, deberá utilizarse una extensión del protocolo BGP.

Capa de transporte: Servicios y primitivas

Esta capa, junto a la capa 3, constituye el corazón de la jerarquía de protocolos OSI.

Introducción

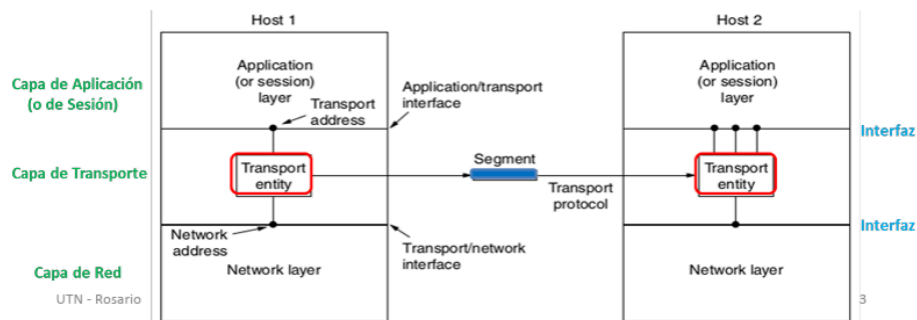
El fin de la Capa de Transporte es brindarle un servicio de transmisión de datos, confiable, eficiente y costo efectivo a la Capa superior de Aplicación.

Entidad de transporte: conjunto de software y hardware que constituye la capa 4. Puede estar localizado en varios lugares:

- en el Kernel del sistema operativo
- en paquetes de librerías con aplicaciones de red,
- en una placa de red,
- en un proceso separado de un usuario.

Observando el modelo OSI desde las capas más bajas, la Capa de Transporte es la primera que opera de “**extremo a extremo**”, sin importar la cantidad de redes que existan en el camino. A diferencia de las capas 1,2 y 3 que trabajan “salto a salto”, o “punto a punto”. Los usuarios o hosts son los principales elementos donde reside el software y hardware de transporte.

Las relaciones lógicas entre la capa de red, de transporte y de aplicación, se observan en la figura:



Servicios

Servicio Orientado a Conexión: Este servicio brinda conexiones de datos confiables, aunque las redes son imperfectas, pierden paquetes, sufren congestiones, etc. Este servicio se encarga de la retransmisión de paquetes para lograr una conexión confiable entre origen y destino. Tiene tres fases bien diferenciadas que son: Establecimiento, Transferencia de datos y Liberación. Y donde también tenemos procesos de control de flujo y direccionamiento de los paquetes. Aquí se utiliza el protocolo **TCP** que realiza control de errores y retransmisión de segmentos si fuera necesario.

Servicio no Orientado a Conexión: El servicio de transporte también puede brindarse sin conexión ni confiabilidad, simplemente como datagramas. Por ejemplo, se emplea en la práctica para streamings de audio y video, donde no hay tiempo para retransmisiones de datos en tiempo real. Es un servicio simple, sin demasiados controles, y que es implementado con el protocolo **UDP**. El control de errores es optativo.

En Capa de Transporte es común ver una arquitectura del tipo “cliente-servidor”

Segmentos

En capa 3 llamamos “paquete” a la unidad de datos, en capa 4 lo denominamos “segmento” al mensaje de capa de transporte.

Primitivas

Las primitivas son unidades de software que realizan una pequeña función específica, como parte del servicio de esa capa. En general, los programadores de la capa de transporte escriben sus primitivas independientemente de la red donde va a funcionar. En capa 4 se las conoce como “**sockets**”.

Primitivas de TCP

En arquitectura cliente-servidor, las 4 primeras primitivas para TCP son ejecutadas en este orden por los servidores:

1. **SOCKET:** Crea un extremo de comunicación y reserva espacio en las tablas de la entidad de transporte.
2. **BIND:** Asigna direcciones locales de capa 4 al socket, para que los clientes se puedan comunicar, (puertos).
3. **LISTEN:** Asigna espacio de almacenamiento para colas de espera de clientes que quieran comunicarse con el server.
4. **ACCEPT:** Se utiliza cuando llega un segmento de cliente solicitando conexión.

Al finalizar la conexión se ejecuta un “CLOSE” de ambos lados. El fin de la comunicación es **simétrico**.

Del lado del cliente, las primitivas son distintas, aunque parecidas. “BIND” no es necesario ya que el servidor no se fija en la dirección del cliente.

Socket orientado a conexión: El socket API se utiliza en TCP para una comunicación “orientada a conexión” llamada “Reliable Byte Stream”.

Socket sin conexión: Los sockets también pueden ser utilizados por aplicaciones para “comunicaciones no orientadas a conexión”. En este caso, la primitiva “CONNECT” define la dirección del lado remoto, “SEND” y “RECEIVE” se encargan de comunicar datagramas entre ambas partes o “peers”.

Características de transporte

Direccionamiento:

En esta capa utilizamos la sigla TSAP (Transport Service Access Point) para indicar un extremo de conexión, o sea el “puerto”. Son 16 bits que brindan 65536 direcciones de transporte. Por lo general, un host utiliza varios puertos en capa 4 a la vez y todos con una sola dirección IP de capa 3. Muchas aplicaciones utilizan puertos conocidos por defecto. Si el puerto destino es desconocido, hay procesos alternativos para averiguarlo:

Port mapper: Es un proceso que asocia el nombre de un servicio al número de puerto correspondiente. Utiliza por defecto el puerto conocido 111 y los hosts lo pueden consultar. Le preguntan qué puerto utiliza un determinado servicio y el port mapper le responde con el número de puerto correspondiente para ese servicio.

Process Server: En otros casos puede utilizarse un process server que monitorea varios puertos a la vez, como un “proxy”. Cuando un cliente busca ese servidor con un determinado puerto, el process server aprovecha ese mismo puerto para conectarlo al servicio en cuestión.

Establecimiento de la conexión

Para establecer una conexión en servicios orientados a la conexión, una entidad de transporte envía un segmento "Connection Request" a una entidad remota y queda esperando un segmento de "aceptación". Suelen aparecer problemas cuando en una red se presentan retardos, pérdidas de paquetes, retransmisiones de segmentos, etc.

Metodos para resolver retardo de segmentos:

Tiempo máximo de vida de un segmento: La mejor forma es limitando la vida de un segmento por tiempo o por saltos, y limitando el tiempo de vida de los "acknowledgements" del receptor. Para evitar segmentos duplicados, se establece un "tiempo máximo de vida del segmento", T , de alrededor de 120 segundos. Entonces se debe esperar un mínimo tiempo T para repetir el segmento con el mismo número de secuencia, sabiendo que los viejos paquetes con esa secuencia ya fueron descartados de la red.

Método de las tres vías, (Three-way handshake): En la práctica se emplea el método de Tomlinson (1975), o método de las 3 vías. Consiste en agregar un número de secuencia en cada segmento, que no puede ser reusado en cierto tiempo T , para garantizar que los segmentos "gemelos" se hayan extinguido. El número de secuencia se extrae de un reloj en tiempo real, que cada host tendrá, y no es necesario que estén sincronizados. El host 2 contesta con un "acknowledgement" de la secuencia usada por host 1, y luego el host 1 responde ya utilizando su secuencia con un "acknowledgement" de la secuencia elegida por host 2.

TCP usa este método para establecer conexiones, con 32 bits de secuencia y un valor inicial aleatorio para evitar predicciones por algún presunto atacante.

Desconexión

Desconexión simétrica: La desconexión es coordinada de ambos lados. En general, la desconexión es más sencilla que la conexión. Pero habría que evitar la desconexión abrupta de un solo lado, ya que podría ocasionar pérdidas de segmentos. La desconexión puede ser simétrica, o asimétrica.

Es simétrica cuando se interpreta la comunicación como dos canales unidireccionales que tienen que desconectarse en forma independiente.

Es asimétrica, en una llamada telefónica, donde si alguno de los extremos cuelga, finaliza la comunicación.

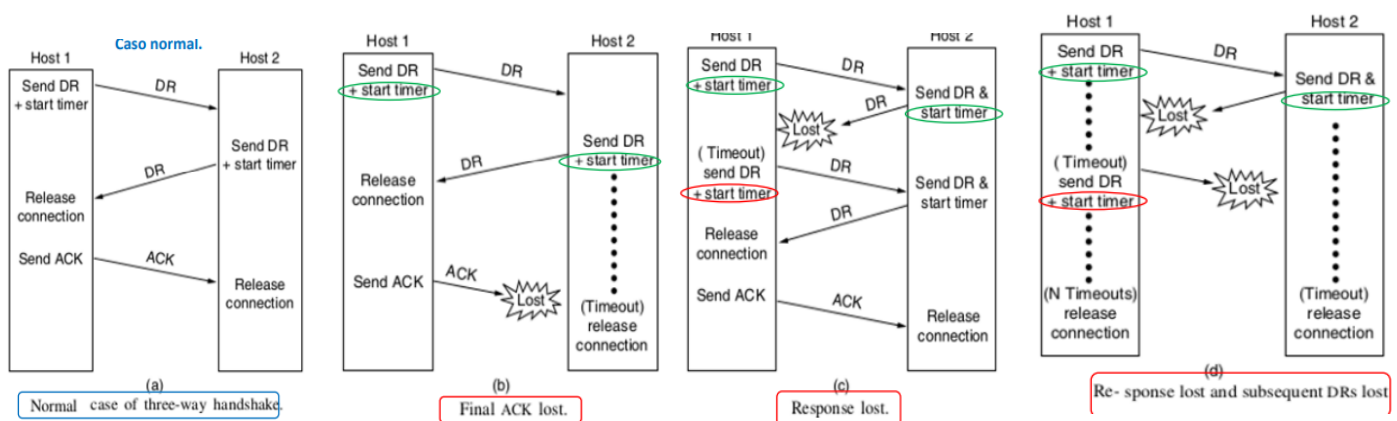
Problema de los dos ejércitos: Si la desconexión es simétrica, existe un problema comparable al "problema de los dos ejércitos". Si el ejército azul ataca en conjunto, vence al ejército blanco. Pero si ataca por separado, el blanco lo supera. El dilema es cómo coordinar las acciones del ejército azul 1 y 2, porque tendrán que enviar mensajeros, que al pasar por el valle podrían ser capturados por el ejército blanco. Supongamos que el ejército azul 1 envía su mensaje y llega con éxito al ejército azul 2. Sin embargo, el ejército 1 no atacará porque no está seguro si el mensaje llegó efectivamente. Supongamos que el mensaje del ejército 1 llega bien, y el ejército 2 envía la confirmación de

que recibió el mensaje al ejército 1. Sin embargo, ahora quién no sabe si atacar o no, es el ejército 2, ya que no sabe si su mensaje ha sido recibido por el ejército 1.

Esta sucesión de mensajes no tiene solución, ya que el último mensaje nunca se sabe si fue recibido por el otro ejército. Esto mismo sucede en la desconexión. No se sabe si el último mensaje fue recibido o no. En la práctica se deja que cada lado decida independientemente cuando desconectar, y por lo tanto, el protocolo para la desconexión coordinada es también un “**handshake de 3 vías**”.

Ejemplos de desconexión:

- Proceso deseado
- Se pierde un acknowledgement, después del timeout se procede a la desconexión
- Se pierde el Disconnection request del 2do host ⇒ el 1ero vuelve a mandar el disconnection request
- Este caso es mas complicado. Se pierde el Disconnection Request del host 2 y también las N replicas del host 1. La desconexión se realiza igual cuando se exceden los tiempos establecidos para ambos hosts



Control de errores y flujo

En capa 4 se emplean los mismos mecanismos de control de errores y de flujo que en capa 2, aunque con distintos grados y funciones.

Control de errores: Al final de cada segmento hay un campo “CRC o checksum” para detectar errores. En TCP es obligatorio, mientras que en UDP es opcional.

Control de flujo: Se realiza con una ventana deslizante. Se establece un número de secuencia para identificar el mensaje y el “acknowledgement” correspondiente. Si no se recibe esta confirmación del receptor, se vuelve a reenviar el segmento después de un

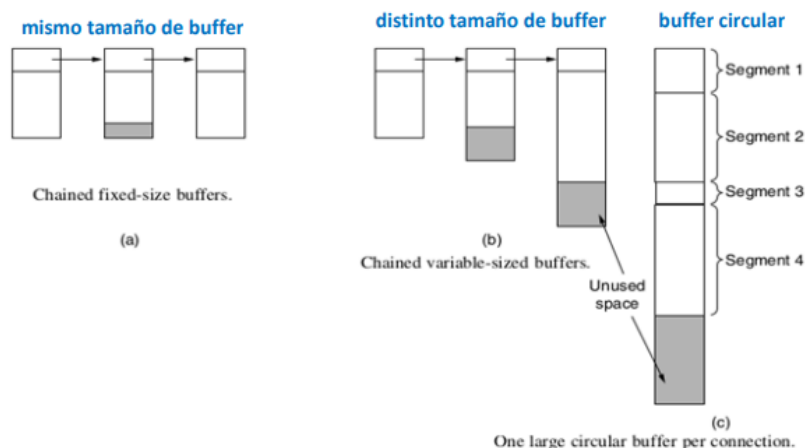
cierto tiempo hasta que el receptor confirme su recepción. Este proceso se conoce como ARQ (Automatic Repeat reQuest).

Observaciones:

- Si se producen errores dentro de un **router**, la capa de enlace no los detectará pero serán detectados por la capa de transporte.
- En cuanto a la ventana deslizante, muchos protocolos inalámbricos, como el estándar WiFi, sólo transmiten una trama y esperan su confirmación, (stop and wait). Una ventana deslizante más grande aumentaría la complejidad sin mejorar la performance. Esto sucede cuando el producto velocidad por retardo es bajo.
- En enlaces cableados, como Ethernet, no se controlan los errores porque es baja la tasa de error y se deja que las retransmisiones extremo a extremo de capa 4 reparen los errores.
- En algunas transmisiones por TCP, que tienen un alto producto "Retardo por Ancho de Banda", se emplea una ventana deslizante mayor para lograr mayor eficiencia.

Por lo general, los protocolos de transporte utilizan ventanas deslizantes grandes, y por lo tanto, analizaremos el problema de los buffers de almacenamiento. Cuando arriba un nuevo segmento, se asigna un nuevo buffer. El mismo tamaño de buffer es lo más sencillo de implementar, aunque también podemos definir tamaños diferentes como en la figura (b), o un espacio circular como en figura (c), que tal vez sea la mejor solución.

Inicialmente, el emisor solicita la reserva de un cierto número y tamaño de cada buffer, según su velocidad y cantidad de datos, y el receptor informa la disponibilidad. Así, con cada "acknowledgement", el receptor informa sobre el número de segmento recibido y el espacio de memoria aún disponible.



Cuando memoria

problema, aparece otro cuello de botella que es la capacidad de la red. Y si el emisor envía más tráfico que el que se puede transmitir, se produce una congestión. Por lo tanto, una **ventana deslizante dinámica** ajusta el flujo a la capacidad de la red y al espacio disponible en memoria.

el espacio de no es el

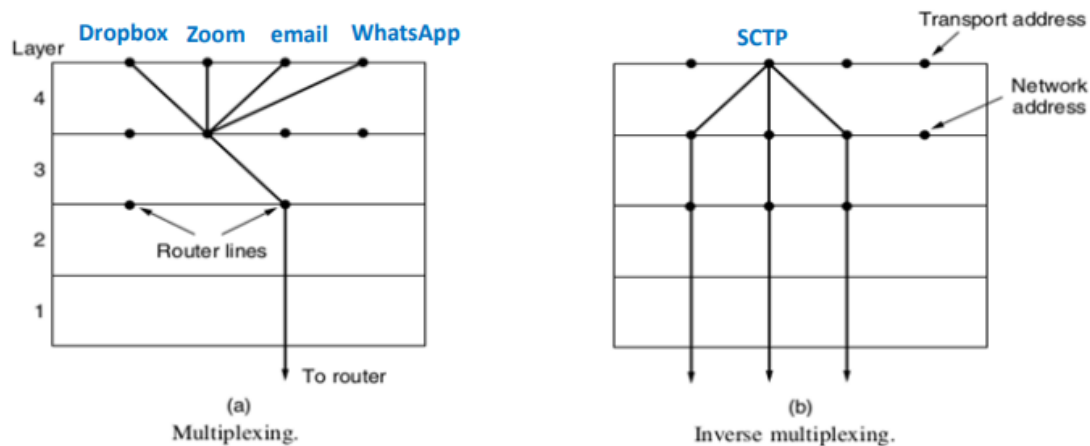
Multiplexacion

En el caso que un host tenga una única dirección de red, todas las conexiones de capa 4 deben compartir ese mismo enlace en el host. En la figura (a) vemos un ejemplo de

multiplexación de una dirección IP en varios puertos, mientras que en la figura (b) se observa un ejemplo de multiplexación inversa.

Un ejemplo de multiplexación inversa es SCTP, (Stream Control Transmission Protocol). Para lograr una velocidad efectiva mayor en el receptor, este protocolo utiliza en paralelo varias conexiones de red disponibles. También hay casos de multiplexación inversa en Capa 2, para mejorar la velocidad de transmisión con múltiples enlaces.

En TCP no se realizan procesos de multiplexación inversa porque es unicast, (un emisor y un receptor).



Control de congestión

El manejo de una congestión es responsabilidad conjunta de las capas de red y de transporte, ya que la congestión ocurre en un router, y por lo tanto es detectada en la capa de red, pero es causada por el tráfico enviado desde la capa de transporte.

Es común que las redes no reserven tasa de bits para las distintas fuentes y que cada una utilice el máximo disponible.

El objetivo del control de congestión es asignar una tasa justa de bits a cada entidad de transporte que comparta la red, de modo de lograr la mejor performance posible sin llegar a congestionar la red.

El mecanismo de control de congestión es quien limita la tasa de bits de cada fuente de transmisión. Decimos que la tasa de bits asignada a una fuente es "Máx-Mín Fair" (Tasa Justa) si cualquier aumento repercute en una disminución de la tasa de bits de otra fuente.

El algoritmo de asignación de tasa de bits debe converger rápidamente a una asignación de Tasa Justa.

Comúnmente las fuentes de tráfico no son constantes, sino que dinámicamente abren y cierran conexiones. Por lo tanto, la asignación correcta de tasa de bits debe ser dinámica y ajustarse periódicamente.

Una forma intuitiva de lograr una asignación de Tasa Justa es que cada fuente comience desde cero a incrementar su tráfico hasta que aparezca una congestión en algún tramo y allí en ese punto no aumente más.

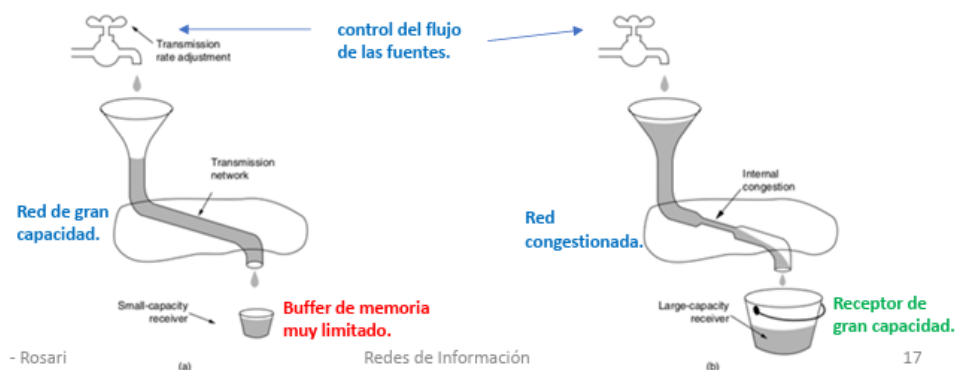
¿Cómo regular el tráfico de cada fuente? → analogía hidráulica

En la figura (a) encontramos una red de gran capacidad conectada a un receptor con buffer de memoria muy limitado. En este caso debe existir un buen mecanismo de control de flujo para que no se pierdan paquetes.

En la figura (b) en cambio, tenemos una red congestionada entregando datos a un receptor de gran capacidad. Debe existir una adecuada asignación de la tasa de bits entre las distintas fuentes.

En ambos casos debemos controlar adecuadamente el flujo de la fuente para no perder datos por provocar una congestión.

- Cada router le explicita a cada fuente el flujo permitido a transmitir.
- La Capa 4 notifica a las fuentes sobre los retardos de cada camino para evitar puntos de congestión.
- En la práctica, la pérdida de paquetes también se utiliza como indicador de congestión.



AIMD (Additive Increase Multiplicative Decrease): Es la ley de control de congestión óptima. Una variación aditiva en la tasa de bits se visualiza como una recta a 45 grados. Una variación multiplicativa se ve como una recta pasando por el origen. AIMD incrementa aditivamente y decrementa multiplicativamente, haciendo que la fuente con mayor tráfico disminuya más velozmente su flujo logrando que el mecanismo converja al punto de operación óptimo. Es la ley de control utilizada en TCP. De todas formas, no es del todo justo ya que TCP ajusta sus ventanas deslizantes según el "round trip time", o tiempo de propagación ida y vuelta, haciendo que las conexiones más cortas tengan más tasa de bits que las conexiones más largas.

Los protocolos de Capa 4 distintos a TCP, deben ser "TCP amigables" y usar una ley de control de congestión similar, ya que de lo contrario podrían apropiarse de la mayor parte de la tasa de bits compitiendo deslealmente.

Crash recovery

Si eventualmente se interrumpe una comunicación por fallas en un router o en un enlace, se producen retransmisiones de los segmentos perdidos. Si en cambio, la falla se genera en el **host receptor**, es más complicado.

Al Enterarse de un “crash” en el receptor, el **emisor** tiene 4 alternativas posibles:

- Retransmitir siempre.
- Retransmitir, aunque no haya un segmento sin su acuse de recibo.
- Retransmitir sólo si hay un segmento sin su correspondiente acuse de recibo.
- No Retransmitir.

Por su parte el host **receptor**, al momento de su fallo podría estar:

- Enviando un “acknowledgement” (A).
- Podría estar escribiendo, guardando un segmento recibido en la memoria (W).

Estrategias:

Según como esté programado el receptor, antes escribirá (W), o antes enviará el acuse de recibo (A) y luego escribirá. Las distintas alternativas se ilustran en la figura, donde la secuencia AC(W), por ejemplo, significa que el receptor envió el “acknowledgement” primero, luego vino el “crash” y, por ende, no pudo escribir el segmento (W) correspondiente.

En algunos casos aparecen segmentos duplicados. En otros, se pierden. Y en algunas combinaciones, pueden perderse o duplicarse segmentos, como se aprecia en la figura.

Strategy used by sending host	Strategy used by receiving host					
	First ACK, then write			First write, then ACK		
	AC(W)	AWC	C(AW)	C(WA)	W AC	WC(A)
Always retransmit	OK	DUP	OK	OK	DUP	DUP
Never retransmit	LOST	OK	LOST	LOST	OK	OK
Retransmit in S0	OK	DUP	LOST	LOST	DUP	OK
Retransmit in S1	LOST	OK	OK	OK	OK	DUP

OK = Protocol functions correctly
 DUP = Protocol generates a duplicate message
 LOST = Protocol loses a message

A = Acknowledgment.
 C = Crash.
 W = Write.

Conclusión:

Nunca será transparente para las capas superiores, cuando se produzca una falla y un posterior restablecimiento del host. Cuánto afecte dependerá de la información que disponga la capa superior.

Cuestiones específicas de redes inalámbricas

Los protocolos de capa 4 deberían ser independientes de la red que subyace en capas inferiores. Sin embargo, hay algunos casos particulares en redes inalámbricas porque suelen tener errores:

WiFi: Un alto “throughput” requiere una tasa baja de pérdida de paquetes. En la práctica, una conexión “fast TCP” tiene una pérdida aceptable de 1%. Sin embargo, las redes inalámbricas pierden una mayor cantidad de datos debido a su propagación por el aire, (10%). Con esta tasa de error, una conexión “fast TCP” dejaría de funcionar. La solución pasa por retransmitir paquetes en capa 2, de forma que sea imperceptible para la capa 4. WiFi utiliza un protocolo “stop-wait” en capa 2, retransmitiendo varias veces en caso de errores antes de notificar a las capas superiores sobre una pérdida de datos.

Redes Satelitales: En redes satelitales, el retardo de ida y vuelta en capa 2 es similar al tiempo límite de retransmisión de capa 4, por lo tanto, en estos enlaces no se retransmite en capa de transporte, sino que se emplea corrección de errores hacia adelante, FEC (Forward Error Correction), mediante códigos correctores.

Protocolo TCP

Este protocolo fue diseñado para brindar una conexión confiable extremo a extremo atravesando redes poco confiables. Se adapta a las distintas topologías de redes, anchos de banda, retardos, tamaño de paquetes y otras variaciones de parámetros que se pueden encontrar en internet.

Una entidad TCP se identifica por un **puerto**, igual que UDP, que es un campo de dirección de 16 bits. Los puertos con número menor a 1024 se reservan para aplicaciones comunes y usuarios con privilegios. Actualmente hay más de 700 puertos asignados. Los puertos, de 1024 en adelante pueden usarse libremente.

Características TCP

TCP comunica dos hosts entre sí, usualmente en arquitectura cliente-servidor. TCP no soporta multicasting o broadcasting. Todas las conexiones TCP son **unicast**, **full-duplex**, de extremo a extremo.

Un segmento TCP puede extenderse hasta casi 64 KB, aunque lo usual es que su tamaño no exceda 1,46 KB para poder ingresar en una trama Ethernet (1500 Bytes) con los encabezamientos de TCP/IP.

Algunas aplicaciones necesitan actualizar datos periódicamente. Para ello, los segmentos TCP tienen un indicador de “**urgente**” para ser enviado lo antes posible, sin esperar el orden del buffer de salida. Y aunque las aplicaciones no pueden forzar a TCP a enviar datos “urgentes”, los sistemas operativos han desarrollado opciones para tal fin.

Una conexión TCP se dice que es “**byte-stream**” y no “mensaje-stream”. Esto quiere decir que, si un proceso escribe 2048 bytes de datos en 4 cadenas de 512 bytes, estos bytes pueden enviarse así tal cual, o en dos cadenas de 1024 bytes, o una sola de 2048 bytes, o alguna otra combinación, el receptor tampoco tiene forma de saber cómo se han particionado los bytes al enviarlos por TCP.

Encabezamiento TCP

La longitud total de un segmento TCP es de casi 64 KB. Tiene la limitación de caber en el campo de carga de un paquete IP que tiene longitud máxima de 64 KB incluido el encabezamiento. Además, en capa 2 tenemos el máximo MTU (maximum transmission unit) que en el caso de Ethernet son 1500 bytes. Si bien puede fragmentarse un segmento IP para entrar en la trama de capa 2, esto trae otros inconvenientes de performance que se busca evitar, por eso la longitud del segmento TCP no suele pasar los 1460 bytes, para no ser fragmentado.

Los primeros campos, llevan los números de los puertos de origen y destino, luego el número de secuencia del segmento enviado y el del segmento recibido.

TCP header Length: El encabezamiento de un segmento TCP consta de 20 bytes fijos más otros bytes opcionales. En el campo de 4 bits que se observa en la figura se indica cuando comienzan los datos, ya que el encabezamiento puede contener algunas opciones que alargan su longitud. Los siguientes 4 bits no se usan (color gris).

CWR y ECE: Indican congestión cuando se utiliza “Explicit Congestion Notification”, (RFC 3168). El bit “ECE = ECN Echo”, le avisa al emisor que reduzca su velocidad de transmisión, y el emisor contesta con “CWR = Congestion Window Reduced”, señalando que ha recibido el mensaje ECE y ha reducido su velocidad.

URG = 1: Se utiliza junto a “Urgent Pointer” para indicar que hay datos urgentes para ser leídos en el receptor y por lo tanto, debe saltarse el orden de secuencia normal en el receptor.

ACK = 1: Significa que el segmento contiene un “acknowledgement” de un segmento recibido.

PSH = 1: Indica que el receptor debe volcar los datos recibidos a la aplicación sin guardarlos en un buffer, (“PuSH”).

RST = 1: “Reset”. Indica que debe recomenzar nuevamente la conexión, debido a algún problema.

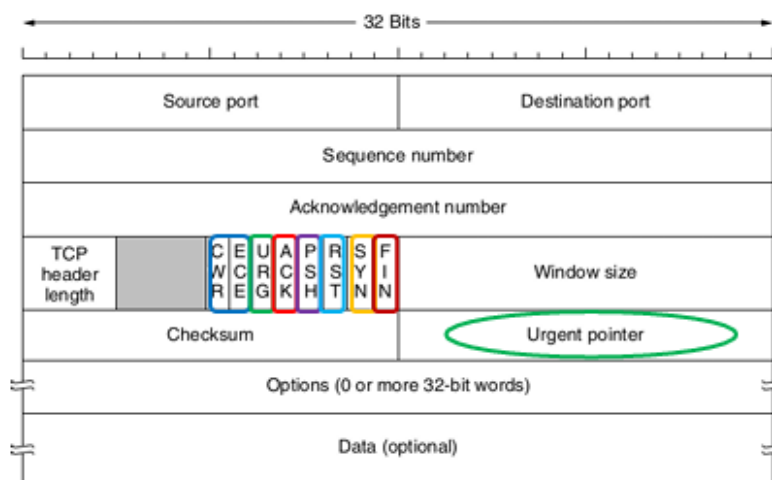
SYN = 1: Se utiliza para sincronizar el establecimiento de la conexión, “Connection Request”.

FIN = 1: Indica que el transmisor no tiene más datos para transmitir y pueden desconectarse, “Disconnection Request”.

Windows Size: Indica el tamaño de la ventana deslizante para control de flujo. Es decir, la cantidad de bytes que pueden enviarse sin recibir su “acknowledgement” correspondiente. El máximo es 64 kB.

Checksum: Es obligatorio en TCP. Controla el segmento completo TCP más el pseudoencabezamiento IP, igual que en UDP. Este detecta errores en el segmento TCP completo y en algunos campos del paquete IP de capa 3, violando la estructura del modelo de capas.

“Options”: Estos campos opcionales añaden facilidades no contempladas en el encabezamiento normal, y pueden extenderlo hasta un máximo de 20 bytes más. Algunas opciones se definen al establecerse la conexión, otras en cambio, pueden definirse en cualquier momento, en medio de los segmentos de información.



Campos opcionales de encabezamiento

Tamaño máximo de segmento: La más usada. Indica el máximo tamaño de segmento permitido. Por defecto es de 556 bytes totales, que debe ser aceptado por cualquier host. El tamaño puede ser diferente en un sentido y en el sentido opuesto de la comunicación. Es determinante en el tamaño del buffer del receptor. (“Maximum Segment Size”).

Tamaño de ventana deslizante: Otra muy utilizada. Agrandar el máximo de Ventana Deslizante, ya que en conexiones de gran velocidad y/o grandes retardos, el máximo por defecto de 16 bits (64 KB) para la Ventana es muy poco. La opción “Window Scale” permite tamaños de ventanas de hasta 230 bytes = 1 GB.

Instante de salida: Agrega el instante de tiempo (“Timestamp”) en que sale el segmento. Es utilizado para estimar el tiempo de viaje del segmento y así poder decidir si se da por perdido y retransmitirlo.

SACK = “confirmación selectiva”: Otra opción muy utilizada. Permite al receptor enviar de una sola vez el rango de secuencia de los segmentos que han sido bien recibidos.

Establecimiento de la conexión

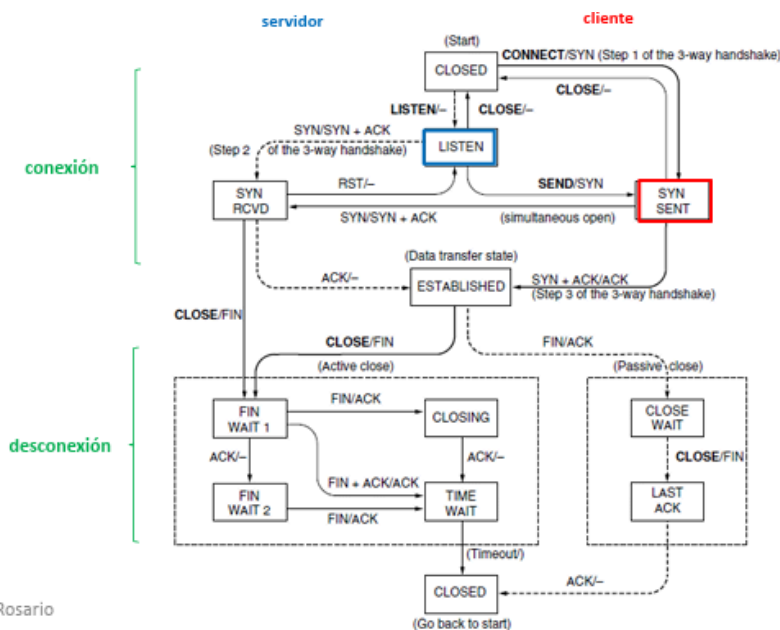
“Handshake de 3 vías”

- El servidor ejecuta la primitiva LISTEN.
- El cliente ejecuta la primitiva CONNECT, enviando la dirección IP y el puerto en destino al que se quiere conectar, el tamaño de segmento máximo que admite, y algún otro dato adicional. Lleva el bit SYN en “1”, el bit ACK en “0”, esperando una respuesta del otro lado.

Por razones de seguridad hoy se emplea criptografía también en estos procesos para evitar conexiones maliciosas.

Secuencia de estados de la comunicación

En la figura se observan los estados por los que pasa una conexión TCP. El “handshake de 3 vías” para la conexión y para la desconexión. Las líneas gruesas indican la secuencia normal de un cliente. Las líneas finas marcan secuencias inusuales. Para la desconexión se necesita desconectar ambos sentidos de la comunicación y, por lo tanto, se envían 4 primitivas, un “FIN” y su “ACK” en cada dirección. Si se pierde uno de estos segmentos la desconexión se efectúa igual con un temporizador que expira su cuenta, evitando así el problema de “los dos ejércitos”.



Temporizadores

Se usan varios. El más importante es el que decide una **retransmisión del segmento**, al considerarlo perdido. Es algo más complicado que en capa 2, porque hay menos desviación estándar en la capa de enlace. La probabilidad de recibir un “acknowledgement” está más distribuida en el tiempo.

Temporizador dinámico: La mejor solución es ajustar el “time-out” en forma dinámica, adaptándolo al estado de congestión de la red. El tiempo mínimo suele ser 1 segundo en capa 4.

Control de congestión

TCP utiliza una “ventana deslizante” definida como una cierta cantidad de bytes que está viajando por la red sin su correspondiente “acknowledgement”. TCP ajusta dinámicamente el tamaño de esta ventana según la regla AIMD (Additive Increment, Multiplicative Decrement)

Los “acknowledgements” recibidos determinan la velocidad adecuada para enviar nuevos segmentos. Por ejemplo, se envía un primer enlace muy rápido seguido de un enlace más lento después del router. Por lo tanto, el emisor envía sólo una primera ráfaga pequeña de segmentos y se detiene a esperar que retornen las confirmaciones del receptor. Este proceso evita una congestión y se denomina “Acknowledgement-Clock”.

Algoritmo de incremento aditivo: Una forma de ajustar el tamaño de la ventana deslizante es aumentar en una unidad su tamaño con cada retardo ida y vuelta.

Algoritmo “slow start”: Este algoritmo es mejor aún. Consiste en aumentar al doble el tamaño de la ventana con cada confirmación del receptor, es decir, midiendo el retardo ida y vuelta. Comienza lento, enviando un primer segmento, y si su acuse de recibo retorna antes del tiempo preestablecido, la ventana de congestión se incrementa al doble, enviando 2 segmentos. Y así sucesivamente. En algún momento habrá congestión y entonces se disparará el “time-out” de retransmisión de algún segmento. Cuando esto suceda, la ventana de congestión se achicará a la mitad.

TCP Tahoe: La conexión TCP comienza en “slow-start” hasta llegar a un cierto tamaño de ventana (Threshold) y luego continúa evolucionando con incrementos aditivos, más suaves. Al recibirse por triplicado un aviso de que falta un segmento, se considera que se ha perdido e indica que hay una congestión. El algoritmo comienza nuevamente pero ahora con un tamaño de ventana deslizante igual a la mitad del anterior.

TCP Reno: Es una versión posterior de TCP y es parecido al anterior TCP Tahoe, sólo que se agrega un “fast recovery” luego de perder un paquete, retomando desde un tamaño de ventana igual a la mitad del tamaño anterior, se utiliza la regla AIMD después de la primera pérdida.

Acknowledgement selectivo

Confirmación selectiva: Gran parte del funcionamiento de TCP pasa por determinar cuando se ha perdido un segmento analizando los “acknowledgements” que van llegando.

Empleando la opción SACK, es sencillo saber qué segmento retransmitir.

Capa de transporte: Protocolo UDP

Encabezamiento UDP

Este protocolo brinda servicios sin conexión. Envía segmentos de puerto a puerto, dejando que las aplicaciones en capas superiores gestionen errores y controlen el flujo.

El encabezamiento UDP consta de 8bytes, 2 bytes se usan para el puerto origen y otros 2 para el destino. La longitud mínima del segmento UDP es 8bytes, mientras que la máxima es 65515bytes.

Si el **checksum opcional** detecta errores, avisa a las capas superiores. Revisa el segmento UDP completo más un pseudoencabezamiento IP. Suma las palabras de 16 bits de longitud en complemento a uno y toma el complemento a uno de esta suma. Entonces, cuando el receptor chequea este checksum, el resultado debe ser 0.

Pseudoencabezado IP

Su inclusión ayuda a detectar paquetes de capa 3 con errores, aunque constituye una violación en la estructura de capas del modelo OSI, porque la capa 4 controla parte de la capa 3. Está formado por direcciones IPv4 de 32 bits, un byte de 0, el número de protocolo UDP (17) y la longitud del segmento UDP.

UDP puede detectar errores con su checksum, pero deja que las capas superiores decidan que hacer, no realiza control de flujo ni retransmisiones. Solamente gestiona puertos distintos para separar distintas comunicaciones. No requiere proceso de conexión ni de desconexión.

Procesos remotos

En capa 4 es común que un host solicite ejecutar un proceso en otro host remoto. Eso se conoce como **RPC (Remote Procedure Call)**. El host 1 es el cliente y el host 2 es el servidor. Ejemplos donde se da esto es solicitar una canción en Spotify o una película en Netflix.

Ejemplo del PDF:

1. El cliente ejecuta localmente una primera parte del proceso, llamado “client stub”. (stub = colilla, cabo).
2. Este proceso solicita el envío del segmento al servidor, “marshaling”.
3. El sistema operativo del cliente envía el segmento.
4. El sistema operativo del servidor pasa el mensaje al proceso correspondiente en el servidor, “server stub”.
5. El “server stub” solicita al servidor propiamente dicho, los parámetros pedidos.

Luego se envía la respuesta al cliente con pasos similares en dirección opuesta.

Transporte de datos en tiempo real

En streaming se usa UDP y RTP (Real Time Protocol).

RTP tiene como función básica juntar varios paquetes de datos en tiempo real y encapsularlos en segmentos UDP. Si se pierde un paquete RTP, es problema de la aplicación. Como no tiene sentido retransmitir, RTP no tiene mecanismos de retransmisión ni envía notificación por cada trama recibida (igual UDP).

RTP (Real Time Protocol)

RTP define varios perfiles de streaming. Cada perfil está pensado para un servicio diferente y permite varias maneras de codificar estos datos. Por ejemplo, una canción en Spotify puede ser codificada como MP3.

Es muy importante el instante de tiempo en que se codifica una muestra, ya que el receptor debe respetar este instante al decodificar los datos, independientemente del retardo que tuvo la red, para que el mensaje sea reproducido tal como fue grabado. Por ejemplo, una película en Netflix tiene varias cadenas de datos relacionados: video, voz en español, voz en inglés, etc; todo eso debe estar sincronizado en el receptor.

Encabezamiento:

- Ver = Version = 2 bits para detallar la versión.
- P = Padding = bit de relleno.
- X = Indica que existe un encabezamiento adicional.
- CC = Indica la cantidad de fuentes de streaming, con 4 bits.
- M = Bit usado por la aplicación en capa 7, por ejemplo, para indicar un comienzo de muestra de audio.
- Payload Type= Son 7 bits que detallan la codificación de los datos, MP3, MP4, etc.
- Sequence number= Número de secuencia de 16 bits, como TPC, para identificar el segmento.
- Timestamp = Instante de inicio del segmento. Importante para mantener el sincronismo y reducir el jitter.
- Synchronization source identifier= Indica a qué fuente de streaming pertenecen los datos transportados.
- Contributing source identifier = Indica las distintas fuentes de streamings que integran una mezcla, si es que existe.

RTCP (Real-Time Transport Control Protocol)

Se encarga de la sincronización y el feedback de la interfaz de usuario en las transmisiones en RTP. La información enviada por RTCP puede ser utilizada para bajar la velocidad (y calidad) cuando hay problemas en la red y viceversa.

El Jitter es la variación del retardo entre varios segmentos. Se estudia para definir cuál es el retardo adecuado.

